01 - Train a custom baseline (U-Net / MedNeXt / DA-VNet / AKS-UNet)

Runtime -> Change runtime type -> **T4 GPU**. Each team member sets `MODEL` to their model. Training checkpoints to Drive every epoch; if Colab disconnects or units run out, **just re-run all cells** and it resumes. Set `MAX_HOURS` a little below your remaining session so it stops cleanly.

```
MODEL = 'mednext'      # 'unet3d' | 'mednext' | 'davnet' | 'aksunet'
SMOKE = True           # True: 3-epoch timing run into runs/smoke_<model>. False: the real run
EPOCHS = 200           # x 250 iterations; set from the smoke-test timing, identical across models
MAX_HOURS = 3.5
PATCH = '192 192 16'   # H W D(slices). On OOM: '160 160 16' or BATCH = 1
BATCH = 2
```

```
# --- shared setup: Drive + code ---------------------------------------------
from google.colab import drive
drive.mount('/content/drive')

DRIVE = '/content/drive/MyDrive/figo'          # shared project folder (one Colab account)
REPO_URL = 'https://github.com/acm-research-f26/Radii.git'  # public; if made private see README (token in Colab Secrets)
REPO = '/content/figo-repo'

import os, subprocess
try:  # private repo: read-only token stored as GITHUB_TOKEN in Colab Secrets
    from google.colab import userdata
    REPO_URL = REPO_URL.replace('https://', f"https://{userdata.get('GITHUB_TOKEN')}@")
except Exception:
    pass  # public repo
os.makedirs(DRIVE, exist_ok=True)
if not os.path.exists(REPO):
    subprocess.run(['git', 'clone', '-q', REPO_URL, REPO], check=True)
else:
    subprocess.run(['git', '-C', REPO, 'pull', '-q'], check=True)
%cd {REPO}
!pip -q install -r requirements.txt
```

```
Mounted at /content/drive
/content/figo-repo
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.7/1.7 MB 32.2 MB/s eta 0:00:00
```

```
!nvidia-smi --query-gpu=name,memory.total --format=csv
# copy the cache to local disk once per session (fast random access during training)
!rsync -a {DRIVE}/data/cache_0.9mm/ /content/cache/
RUN = f'{DRIVE}/runs/' + (f'smoke_{MODEL}' if SMOKE else MODEL)
if SMOKE:
    EPOCHS = 3
```

```
name, memory.total [MiB]
Tesla T4, 15360 MiB
```

```
print(RUN)
```

```
/content/drive/MyDrive/figo/runs/smoke_mednext
```

```
!nvidia-smi
import importlib
for name in ('tensorflow', 'torch', 'jax'):
    try:
        m = importlib.import_module(name); print(name, m.__version__)
    except Exception:
        print(name, 'not installed')
try:
    import tensorflow as tf; print('TF GPUs:', tf.config.list_physical_devices('GPU'))
except Exception as e: print('TF:', e)
try:
    import torch; print('torch cuda:', torch.cuda.is_available())
except Exception as e: print('torch:', e)
try:
    import jax; print('jax devices:', jax.devices())
except Exception as e: print('jax:', e)
```

```
Sun Oct 18 22:59:47 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   53C    P0             27W /   70W |     107MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            1697      C   /usr/bin/python3                        104MiB |
+-----------------------------------------------------------------------------------------+
tensorflow 2.21.0
torch 2.11.0+cu130
jax 0.11.1
TF GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
torch cuda: True
jax devices: [CudaDevice(id=0)]
```

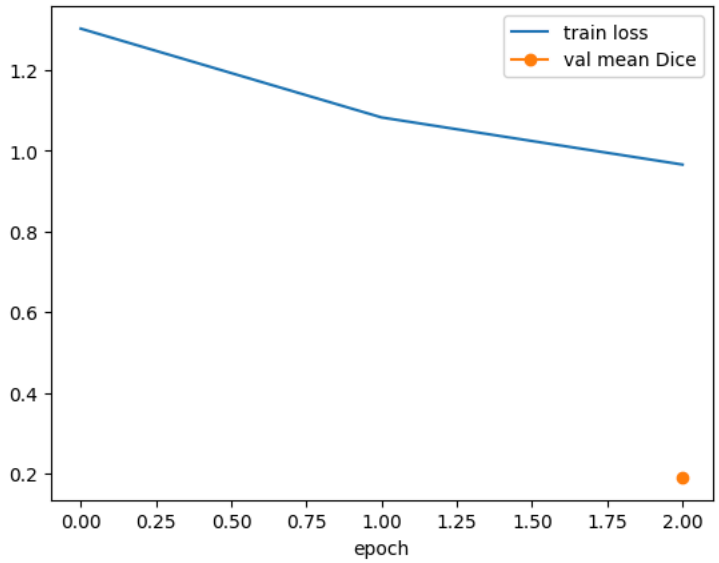
```
!python -m figo.train --model {MODEL} --cache-dir /content/cache --splits {DRIVE}/data/splits.json \
    --out-dir {RUN} --epochs {EPOCHS} --max-hours {MAX_HOURS} --patch-size {PATCH} --batch-size {BATCH}
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1792364404.061429    4723 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
Failed to load nvtiff library! Please install nvTIFF (see https://docs.nvidia.com/cuda/nvtiff/userguide.html#installing-nvtiff).
Note: If using nvImageCodec's Python distribution, it is enough to install the nvTIFF wheel: e.g. `python3 -m pip install nvidia
[ERROR] [nvtiff_ext] Failed to load nvTIFF library or retrieve version. Decoders and encoders will not be registered.
Failed to load nvjpeg2k library! Please install nvJPEG2000 (see https://docs.nvidia.com/cuda/nvjpeg2000/userguide.html#installin
Note: If using nvImageCodec's Python distribution, it is enough to install the nvJPEG2000 wheel: e.g. `python3 -m pip install nv
[ERROR] [nvjpeg2k_ext] Failed to load nvJPEG2000 library or retrieve version. Decoders and encoders will not be registered.
Loading 210 training volumes ...
mednext: 5.5M parameters, deep supervision=True
epoch=0 lr=1.00e-02 train_loss=1.3019 epoch_sec=191
epoch=1 lr=6.94e-03 train_loss=1.0819 epoch_sec=192
epoch=2 lr=3.72e-03 train_loss=0.9652 val_wall=0.3543 val_cavity=0.2436 val_myoma=0.1641 val_cyst=0.0000 val_mean=0.1905 val_ema
Done. Best smoothed val Dice 0.1905
```

```
# timing -> how many epochs fit the budget (row 1 = pure training, row 2 = training + validation)
import csv
log = list(csv.DictReader(open(f'{RUN}/log.csv')))
for r in log:
    print('epoch', r['epoch'], r['epoch_sec'], 's')
if len(log) >= 3:
    t_train = float(log[1]['epoch_sec'])
    t_val = max(float(log[2]['epoch_sec']) - t_train, 0)
    per_epoch = t_train + t_val / 10          # validation runs every 10 epochs
    print(f'~{per_epoch:.0f} s per epoch on average; validation ~{t_val:.0f} s')
    for h in (6, 8, 10):
        print(f'  {h:2d} GPU-hours  ->  EPOCHS = {int(h * 3600 / per_epoch)}')
!nvidia-smi --query-gpu=name,memory.used,memory.total --format=csv
```

```
epoch 0 191 s
epoch 1 192 s
epoch 2 241 s
~197 s per epoch on average; validation ~49 s
   6 GPU-hours  ->  EPOCHS = 109
   8 GPU-hours  ->  EPOCHS = 146
  10 GPU-hours  ->  EPOCHS = 182
name, memory.used [MiB], memory.total [MiB]
Tesla T4, 107 MiB, 15360 MiB
```

```
# learning curves
import csv, matplotlib.pyplot as plt
log = list(csv.DictReader(open(f'{RUN}/log.csv')))
plt.plot([float(r['train_loss']) for r in log], label='train loss')
v = [(int(r['epoch']), float(r['val_mean'])) for r in log if r['val_mean']]
if v: plt.plot(*zip(*v), 'o-', label='val mean Dice')
plt.legend(); plt.xlabel('epoch'); plt.show()
```



Test-set inference and evaluation (run after training has finished)

```
PRED = f'{DRIVE}/preds/{MODEL}'
RES = f'{DRIVE}/results'
!python -m figo.infer --checkpoint {RUN}/best.pt --data-dir {DRIVE}/data/clean --splits {DRIVE}/data/splits.json \
    --subset test --out-dir {PRED}
!python -m figo.evaluate --pred-dir {PRED} --data-dir {DRIVE}/data/clean --splits {DRIVE}/data/splits.json \
    --subset test --out {RES}/{MODEL}_seg.csv
!python -m figo.figo_eval agreement --ref-dir {DRIVE}/data/clean --pred-dir {PRED} \
    --splits {DRIVE}/data/splits.json --subset test --out {RES}/{MODEL}_figo_agreement.csv
```

```
[28/60] UMD_221129_122
[29/60] UMD_221129_123
[30/60] UMD_221129_125
[31/60] UMD_221129_127
[32/60] UMD_221129_129
[33/60] UMD_221129_134
[34/60] UMD_221129_138
[35/60] UMD_221129_150
```

```
assd_wall                        21.680 +/- 23.193
dice_cavity                       0.277 +/- 0.178
hd95_cavity                      75.944 +/- 44.314
assd_cavity                      28.716 +/- 28.170
dice_myoma                        0.292 +/- 0.248
hd95_myoma                       67.244 +/- 50.828
assd_myoma                       29.756 +/- 35.060
dice_cyst                         0.000 +/- 0.000
hd95_cavity_wall_interface       71.455 +/- 45.307
assd_cavity_wall_interface       27.276 +/- 28.337
hd95_serosa                      66.150 +/- 43.711
assd_serosa                      21.841 +/- 22.488
dice_uterus                       0.589 +/- 0.225
macro dice wall cavity myoma      0.326
```